Name: 張哲語
Student ID: 109062336

```
00000020  _unused_thread_count              preemptive
00000021  _car_idx_counter                 preemptive
00000022  _time_counter                    preemptive
00000023  _time_units_elapsed              preemptive
00000024  _empty_slots_cnt                 testparking
00000025  _spot1                           testparking
00000026  _spot2                           testparking
00000027  _spots_mutex                     testparking
00000028  _print_mutex                     testparking
00000030  _stack_pointers_for_threads      preemptive
00000034  _bitmap_for_threads              preemptive
00000035  _tmp                            preemptive
00000036  _created_thread_ID               preemptive
00000037  _current_thread_ID               preemptive
00000038  _available_time                  preemptive
0000003C  _idx_for_threads                 preemptive
```

Delay() and now()

```
#define delay(n) {\
    available_time[current_thread_ID] = time_units_elapsed + n;\
    while (1) {\
        if (time_units_elapsed == available_time[current_thread_ID]) break;\
    }\
}
```

for thread x, use a variable available_time[x] to store the exact time unit that delay is over. So this thread must be stuck until time unit :(current time + n) since delay(n) is called.

```
time_counter += 1;
if (time_counter == 10) {
    time_units_elapsed += 1;
    time_counter = 0;
}
```

I set the time unit as 10 times of the 8051 timer. So time_counter is added 1 in every interrupt. And time_units_elapsed will be added 1 when time_counter reaches 10.

```
unsigned char now() {
    return time_units_elapsed;
}
```

And for now() function, just return the variable time_units_elapsed.Although I didn't call this function in my implementation of the parking lot.

Thread termination and creation.

For thread termination and creation, I used a semaphore "unused_thread_count" to track the number of unused threads.

```
void ThreadExit(void) {
        /*
         * clear the bit for the current thread from the
         * bit mask, decrement thread count (if any),
         * and set current thread to another valid ID.
         * Q: What happens if there are no more valid threads?
         */
        //__critical{
        EA = 0;

        //bitmap_for_threads ^= 1 << current_thread_ID;
        if (current_thread_ID == 0) bitmap_for_threads -= 1;
        else if (current_thread_ID == 1) bitmap_for_threads -= 2;
        else if (current_thread_ID == 2) bitmap_for_threads -= 4;
        else if (current_thread_ID == 3) bitmap_for_threads -= 8;

        SemaphoreSignal(unused_thread_count);
        while (unused_thread_count == 4) {}

        do {
                current_thread_ID += 1;
                if (current_thread_ID == MAXTHREADS) current_thread_ID = 0;

                 if (current_thread_ID == 0) {
                   if (bitmap_for_threads & 1) break;
                 } else if (current_thread_ID == 1) {
                   if (bitmap_for_threads & 2) break;
                 } else if (current_thread_ID == 2) {
                   if (bitmap_for_threads & 4) break;
                 } else if (current_thread_ID == 3) {
                   if (bitmap_for_threads & 8) break;
                 }

        } while (1);

        RESTORESTATE;
```

So upon termination, we add 1 to the semaphore. If there are no existing threads, we stuck this thread in a while loop until another thread is created to avoid bugs. In addition, ThreadExit() also needs to update bitmap and try to find the next running thread if one exists. Finally, restore the registers for the next running thread.

```
SemaphoreWait(unused_thread_count);
ThreadCreate(Park);
SemaphoreWait(unused_thread_count);
ThreadCreate(Park);
SemaphoreWait(unused_thread_count);
ThreadCreate(Park);
SemaphoreWait(unused_thread_count);
ThreadCreate(Park);
SemaphoreWait(unused_thread_count);
ThreadCreate(Park);
```

Cannot create a thread until there is a unused one.

Parking lot example

```
void Park(void) {
        SemaphoreWait(empty_slots_cnt);
        if (spot1 == 0) {
            SemaphoreWait(spots_mutex);

            EA = 0;
            spot1 = idx_for_threads[current_thread_ID];
            EA = 1;

            SemaphoreWait(print_mutex);
            EA = 0;
            print(spot1,'i');
            EA = 1;
            SemaphoreSignal(print_mutex);


            SemaphoreSignal(spots_mutex);

            if (idx_for_threads[current_thread_ID] == 1)
            {
            delay(2);
            }
            else if (idx_for_threads[current_thread_ID] == 2)
            {
            delay(2);
            }
            else if (idx_for_threads[current_thread_ID] == 3)
            {
            delay(5);
```

```
    }
    else if (idx_for_threads[current_thread_ID] == 4)
    {
    delay(2);
    }
    else if (idx_for_threads[current_thread_ID] == 5)
    {
    delay(1);
    }
    //124260


    SemaphoreWait(spots_mutex);

    SemaphoreWait(print_mutex);
    EA = 0;
    print(spot1, 'o');
    EA = 1;
    SemaphoreSignal(print_mutex);

    EA = 0;
    spot1 = 0;
    EA = 1;

    SemaphoreSignal(spots_mutex);
}
else if (spot2 == 0) {
```

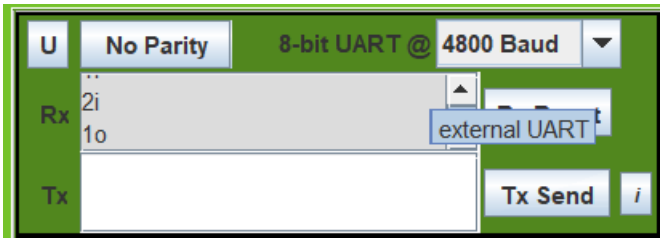
```
    SemaphoreWait(spots_mutex);

    EA = 0;
    spot2 = idx_for_threads[current_thread_ID];
    EA = 1;

    SemaphoreWait(print_mutex);
    EA = 0;
    print(spot2, 'i');
    EA = 1;
    SemaphoreSignal(print_mutex);

    SemaphoreSignal(spots_mutex);


    //delay(2);

    if (idx_for_threads[current_thread_ID] == 1)
    {
    delay(2);
    }
    else if (idx_for_threads[current_thread_ID] == 2)
    {
    delay(2);
    }
    else if (idx_for_threads[current_thread_ID] == 3)
    {
    delay(5);
    }
    else if (idx_for_threads[current_thread_ID] == 4)
```

```
        {
        delay(2);
        }
        else if (idx_for_threads[current_thread_ID] == 5)
        {
        delay(1);
        }
        //124324
        SemaphoreWait(spots_mutex);

        SemaphoreWait(print_mutex);
        EA = 0;
        print(spot2, 'o');
        EA = 1;
        SemaphoreSignal(print_mutex);

        EA = 0;
        spot2 = 0;
        EA = 1;

        SemaphoreSignal(spots_mutex);
    }


    SemaphoreSignal(empty_slots_cnt);
    ThreadExit();
}
```

Use a semaphore empty_slots_cnt(initialized to 2) to avoid multiple cars go in the parking lot at the same time. If successfully find one empty slot, the thread waits until it gets the spots_mutex semaphore(initialized to 1) to update the parking lot slot. Then print ‘i’ for in and ‘o’ for out in the serial port. Ex: ‘3i’ means car 3 is parked into the parking lot. And the printing process is also protected by a semaphore. Then run the delay function for each car to simulate the parking process. I set the parameters of delay function as 2, 2, 5, 2, 1 for car 1, 2, 3, 4, 5. After delay process, update the slots to 0 to indicate that slot is currently empty. Also, update the semaphores and then terminate the thread.

```
#define print(a, b)\
        TMOD |= 0x20;\
        TH1 = -6;\
        SCON = 0x50;\
        TR1 = 1;\
        SBUF = a + '0';\
        while (!TI){}\
        TI = 0;\
        SBUF = b;\
        while (!TI){}\
        TI = 0;\
        SBUF = '\n';\
        while (!TI){}\
        TI = 0;\
```

Codes for printing



Output screenshot